

E³-CODE: Embedded and Efficient Error-Correcting Code for Error-Resilient Neural Networks

Abstract

Neural networks are widely deployed at the edge to process high-dimensional sensor data, but they are susceptible to burst errors that can corrupt weights and degrade inference accuracy. Conventional error-correcting codes (ECC) mitigate errors but incur significant memory overhead. Recent ECC methods for neural networks overwrite the least significant bits of the model weights with parity bits, providing zero-overhead resilience at the expense of slightly reduced inference accuracy. In this paper, we propose a framework for Embedded and Efficient Error-Correcting Code for Error-Resilient Neural Networks called (E³-CODE). The proposed method embeds multi-bit parity within the *entire weight representation*, which is different from only modifying the LSBs of the weights. To minimize the negative impact from the parity embedded ECC, weight and parity assignments are jointly optimized via a mixed-integer linear programming (MILP) formulation. We also propose a hybrid ECC scheme that combines the embedded ECC with conventional ECC to trade-off minor memory overhead for significantly improved resilience. The experimental evaluation on the ImageNet and CIFAR-10 datasets using ResNet, MobileNetV2, and EfficientNet-B0 demonstrates that E³-CODE maintains software-level accuracy in the presence of burst errors with varying Bit Error Rates (BERs). Compared with the best prior methods, the lifetime of an edge system is extended by 4.3X with no memory overhead and 10X with less than 2% memory overhead.

1 Introduction

Neural networks are increasingly deployed at the edge in the Internet of Things (IoT) for traffic control [27, 42], health monitoring [16, 40], and surveillance [11, 28]. They enable real-time processing of sensor data, a critical capability for edge applications. The models are heavily pruned [12], compressed [17], and quantized [9, 22, 44] to meet strict power and memory budgets and maximize battery life. A crucial challenge for edge devices deployed in environments with diverse operating conditions is to provide high resilience to memory errors, which can detrimentally impact the neural network inference accuracy [19, 24, 26].

Memory faults range from transient soft errors [3, 32, 35] to device-level wear, but the dominant threat for tightly scaled SRAM/DRAM and commodity NVMs are burst errors and multi-bit upsets caused by voltage droops, radiation, or coupling effects [3, 29]. Error control spans from simple parity detection using Hamming codes [13] to stronger block codes for multi-bit resilience including Reed–Solomon (RS) [30, 31], Low-Density Parity-Check (LDPC) [6, 8], and Bose–Chaudhuri–Hocquenghem (BCH) [5, 34, 39] codes. All of these methods compute a set of parity bits by computing the XOR of groups of data (or message bits). Next, errors can be detected and corrected using the parity bits. BCH codes are well known for efficiently mitigating burst errors. However, the storage of parity bits can double the memory required

to store the neural network weights, which is prohibitive for edge devices with strict constraints on the hardware and cost.

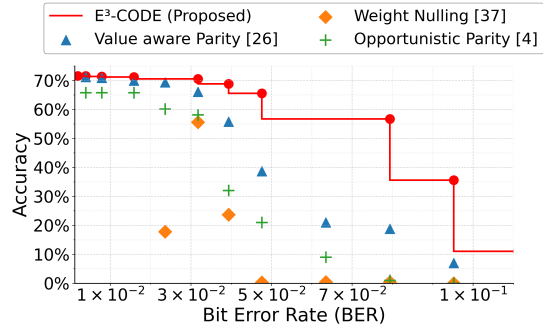


Figure 1: Comparison of E³-CODE with existing ECC techniques [4, 25, 36] in terms of robustness on ImageNet [10] dataset using a ResNet18 [18] model.

The stringent hardware budgets of edge devices have motivated investigations into low-overhead ECC schemes tailored to neural networks [1, 4, 15, 25, 36]. Unlike traditional workloads where application data must be exact, neural network weights can tolerate small perturbations, creating opportunities for resilience–efficiency trade-offs. Broadly, ECC approaches fall into (i) learning-based [1, 15] and (ii) post-training techniques [4, 25, 36]. Post-training methods are generally considered more attractive since they avoid altering the training process or increasing its complexity. Single-bit error mitigation techniques include Weight Nulling [36] that attempts to set weights to zero when errors are detected and Opportunistic Parity [4] that fixates the parity bits to zero by altering the weights. Multi-bit ECC schemes have been investigated in [1, 19, 25]. Selectively protecting the most significant bit (MSB) of the neural network weights was proposed in [20, 24]. Recently proposed Value aware parity [25] overwrites the least significant bits (LSBs) of the weights with the parity bits. While overwriting the LSB with parity bits provides high resilience to errors at zero-memory overhead, the weight distortions negatively impacts the neural network inference accuracy even when no errors are present.

In this paper, we propose an Efficient Error-Correcting Code framework called E³-CODE for error resilient deployment of neural networks on the edge. Compared to prior schemes that overwrite the LSBs of the weights with parity bits (and restrict weight modifications to only the LSBs), our proposed approach allows *all weight bits* to be modified. This facilitates co-optimization of the embedding of weight (data) and parity bits, which leads to advantageous resilience trade-offs, as shown in Figure 1. The main contributions of the paper are summarized, as follows:

- We propose a method for embedding weight and parity bit information while minimizing the effective change of the weights. The co-optimization is formulated and solved using a mixed integer linear program (MILP).

- We propose a hybrid ECC scheme that combines the proposed embedded ECC with regular ECC to balance resilience and memory overhead. Sensitivity analysis is used to identify critical layers for protection using conventional ECC.
- Compared with state-of-the-art frameworks, E³-CODE sustains accuracy while handling bit error rates (BER) up to 10%, whereas existing schemes typically fail beyond a critical BER of around 1%.

The remainder of the paper is organized as follows: preliminaries are given in Section 2 and the methodology is presented in Section 3. The paper is concluded with experimental results and future work in Section 4 and Section 5, respectively.

2 Preliminaries

In this section, we first briefly review fundamentals of protecting on-device memory using ECC. This is followed by a review of related work and a motivational example.

2.1 Baseline Error Correction Codes

Traditional single-bit ECC like Hamming codes [13], provide limited protection against burst errors. Multi-bit error-correcting schemes include LDPC [6, 8], RS [30, 31] and BCH [5, 34, 39] codes. BCH codes are a popular ECC scheme due to their ability to efficiently correct a variable number of errors. A BCH code is denoted as (n, k, t) , where n is total codeword length, k is the number of data bits, and t is the maximum number of correctable bit errors. The BCH code $(7, 4, 1)$ encodes $4 = (7 - 3)$ data bits with 3 parity bits while providing error correction capability of up to 1 bit per codeword. Each parity bit is computed by applying the XOR operation to a subset of the message bits as shown in Figure 2(a). These equations can be captured, as follows,

$$(Am^T)_2 = p^T \quad (1)$$

where A is a binary matrix where each row contains a parity constraint. The matrix A is a generator polynomial matrix obtained from a finite field $GF(2^m)$ [34], m and p are vectors of data and parity bits, respectively. $(\cdot)_2$ is the modulus 2 operator. The BCH code can correct up to one erroneous bit per seven message bits, handling a BER of $t/n = 1/7 = 0.14$. Once the parity bits are computed, error checking is performed by computing the syndrome $S = (A \cdot m^T)_2 \oplus p^T$. If all syndrome bits are zero, no error is detected. Otherwise, the non-zero syndrome is used to index a lookup table that identifies the erroneous bit location, which is then flipped to correct the error, as shown in Figure 2(b).

2.2 Related Works on ECC for Neural Networks

We compare our proposed approach with existing methodologies for protecting neural networks against memory errors using ECC in Table 1. The comparison is performed in terms of memory overhead, fault tolerance, accuracy degradation and need for re-training. Fault tolerance is a measure of the number of errors that the method mitigate. Accuracy degradation is the degradation of the nominal inference accuracy induced by modifying weights to incorporate ECCs. Conventional ECC is robust but introduces significant memory overhead. Techniques such as Weight scaling [14] and Selective protection [20] provide limited fault tolerance. Techniques such as

Table 1: Comparison of ECC schemes for neural networks in terms of overhead, fault tolerance, accuracy, and retraining.

Scheme	Memory overhead	Fault tolerance	Accuracy degradation	Re-training
Conventional ECC [2]	High	High	Low	No
Weight-scaling ECC [14]	Low	Low	Medium	No
NN-ECC [1]	Low	High	Low	Yes
Weight throttling [15]	Low	Medium	Medium	Yes
Selective protection [20]	Low	Medium	Medium	No
Weight Nulling [36]	Low	Low	High	No
Opportunistic Parity [4]	Low	Medium	High	No
Value-Aware Parity [25]	Low	Medium	Medium	No
E ³ -CODE (Proposed)	Low	High	Low	No

Weight nulling [36] works by detecting corrupted weight bits and setting those erroneous weight bits to zero to prevent ECC codes to introduce large errors during the correction process.

A few recent studies have modified the neural network weights to incorporate parity information [1, 25, 36]. The average weight distortion D introduced by incorporating the parity can be measured using the following equation:

$$D = \frac{1}{W} \sum_{i=1}^W |w_i - w_i^{eff}|, \quad (2)$$

where W is the total number of weights. w_i and w_i^{eff} is the weight i before and after parity information has been incorporated. The drawback of this approach is that weight distortion may significantly degrade the inference accuracy of heavily quantized neural networks. To mitigate the accuracy loss, re-training based methodologies were proposed in NN-ECC [1] and Weight Throttling [15]. However, re-training remains computationally expensive for large-scale models and it is ideal to decouple ECC from the training process. On the other hand, methods like Opportunistic parity [4] fixes all the parity bits into a fixed value zero by altering the weight bits, but introduces significant weight distortions. Finally, Value-Aware Parity [25] addresses the multi-bit errors by overwriting multiple LSBs with parity bits. This enables error correction without retraining, but the resulting weight distortion makes the scheme break down at higher BER. In contrast, our proposed E³-CODE scheme of co-embedding weights and parity bits results in high fault-tolerance with small accuracy degradation.

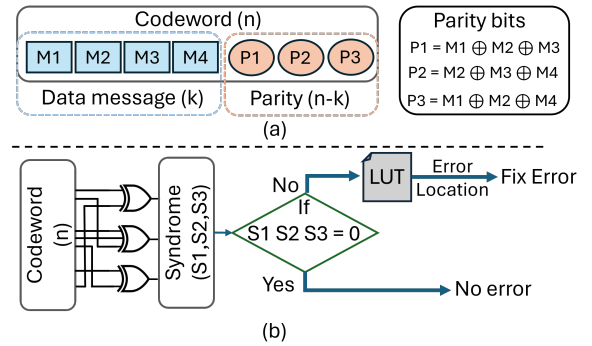


Figure 2: (a) Codeword of a (7,4,1) BCH code. (b) Decoding process with the BCH code.

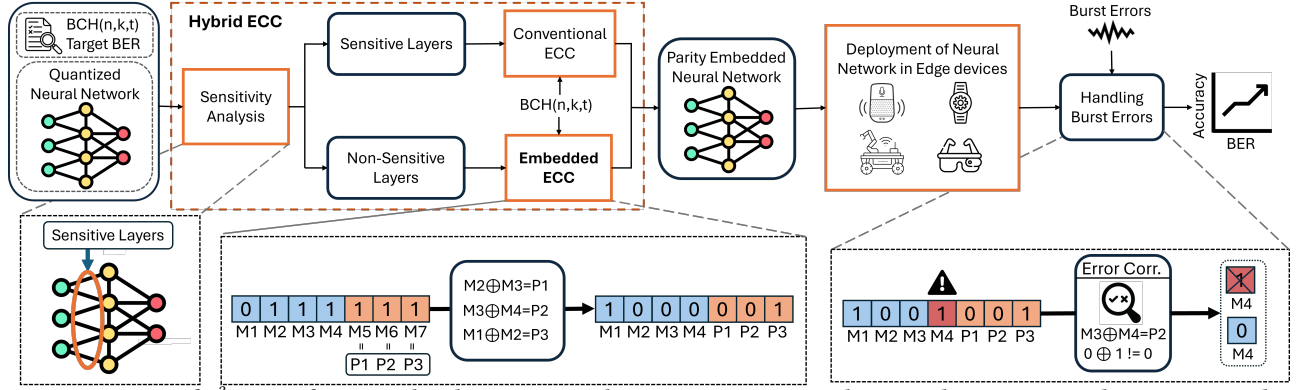


Figure 3: Overview of E³-CODE framework, where sensitive layers use conventional ECC and non-sensitive layers use MIP-based embedded ECC in quantized weights for burst-error-resilient edge devices deployment.

2.3 Motivational Example

In this motivational example, we examine the limitations of simply overwriting the LSBs of neural network weights with parity bits. Consider protecting a 7-bit weight using a BCH (7, 4, 1) code, which is shown in the top of Figure 4. The original value of the weight is 32. Next, the four MSBs of the weight are used to compute the 3 parity bits. Using the parity bit equations, the parity bits P3, P2, and P1 are computed to be 1, 1, 1, respectively. When overwriting the 3 LSBs (M5, M6, and M7), the new effective weight becomes 39, i.e., a weight distortion of magnitude $|32 - 39| = 7$ is introduced. In contrast to naively overwriting the LSBs, we consider all bits of the weight to be modifiable. Next, consider setting M1 through M7 to 0011101, which is shown at the bottom of Figure 4. It can be observed that $M5 = P1$, $M6 = P2$, and $M7 = P3$, which is required for protecting the weight against memory errors. Moreover, the new weight value is 29, which only introduces an error of magnitude $|32 - 29| = 3$. This showcases that there is an opportunity to reduce the weight distortions by co-embedding all weight bits and parity bits. Systematically performing the co-embedding of the weights and parity information is the core contribution of our methodology.

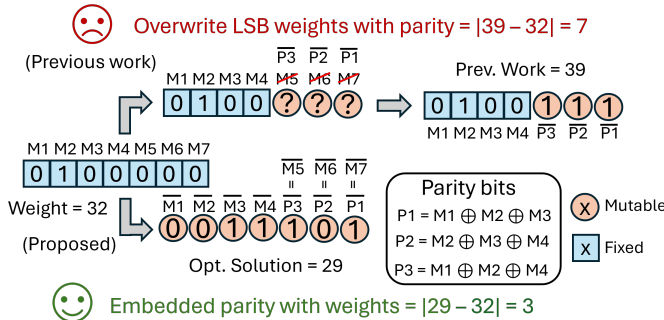


Figure 4: (top) Previous work overwrites the LSBs of the weights with parity bits information. (bottom) The proposed method allows all the weight bits to be modified, which reduces the distortion of the weight value.

3 The E³-CODE Methodology

In this section, we describe the methodology of the proposed E³-CODE framework. An overview of the E³-CODE framework is illustrated in Figure 3. The framework aims to protect a neural network against memory errors with minimal memory overhead and nominal accuracy degradation. The input to the framework is the desired BCH code (n, k, t) , where n is governed by the hardware and the required error tolerance t defined by the operating environment. k is implicitly defined by n and t . The input also consists of a neural network model consisting of L layers. The E³-CODE framework first performs a sensitivity analysis to determine which of the neural network layers will be protected using conventional ECC [2] and proposed embedded ECC, respectively. The details of the hybrid ECC protection are provided in Section 3.2. The implementation of the conventional ECC is straightforward and details can be found in [2]. The embedded ECC approach involves modifying the data (weights) such that the LSBs of the weights are equal to the parity bits. The decomposition of the neural network weights into messages and the co-optimization of each message is described in Section 3.1. We introduce the embedded ECC technique before the hybrid ECC, as it is the core contribution of the paper. Finally, the protected weights are loaded onto an IoT device and the inference accuracy is measured in the presence of burst errors.

3.1 Embedded ECC

In this section, we explain how the embedded ECC is applied to neural network layers containing weights parameters such as convolutional layers (full, expand, depth, projection) and fully-connected (FC) layers. The weights of each layer are divided into groups of size equal to the BCH codeword size n . We represent each weight using h unsigned bits and use a negative offset to construct negative numbers [33, 45]. For example, consider a BCH code with $n = 63$ bits and weights encoded using $h = 8$ bits. In this case, seven full weights ($7 \cdot 8 = 56$ bits) together with 7 additional weight bits form a 63-bit group. Hence, it can be observed that weights may be split between two adjacent codewords. For each of these groups, we formulate and solve a separate optimization problem to embed the parity bits of BCH code within the weights. The objective is to minimize the distortion of the weights while ensuring that the codeword

satisfies the parity constraint in Eq (1). We begin by introducing our notation before formulating and solving the co-embedding using an MILP formulation.

Our approach will store n bits of weights within a codeword of length n . To formulate this mathematically, we introduce the following notation. Without loss of generality, assume that the code word length n is divisible by the number of bits used to represent each weight h . Let the number of weights stored in a codeword be $W = n/h$. Next, let m_i with i in 1 to n denote the individual bits of the codeword. The message bits are the decision variables of our optimization problem. Let w_i^j denote the bit j of the target weight i , where bit i has a value of 2^j .

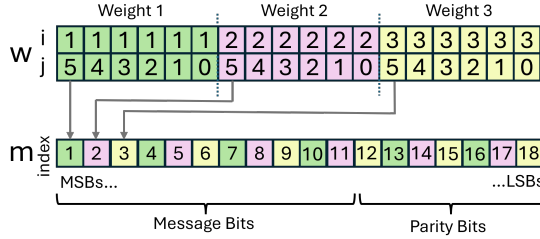


Figure 5: Conversion of variables between the weight representation indices to codeword indices.

Now we explain the mapping of the message bits into the weight representation space, which is illustrated in Figure 5. Given W weights represented using h bits each, the weights are organized from the MSBs to the LSBs in an interleaved fashion as shown in the figure. (Note that this data organization is for optimization and is decoupled from the data layout used in hardware.) This data organization results in that weight bit w_i^j will be placed at codeword bit $m_{i+(h-j+1) \cdot W}$. Moreover, let $\mathbf{m}^1 = [m_1, \dots, m_{n-k}]$ be the message part of the code word and $\mathbf{m}^2 = [m_{n-k+1}, \dots, m_n]$ be the parity portion. Next, we formulate an optimization problem, as follows:

$$\min \sum_{i=1}^W |w_i - w_i^{eff}| \quad (3)$$

$$\text{s.t. } w_i^{eff} = \sum_{j=1}^h 2^j \cdot m_{i+(h-j+1) \cdot W} \quad i = 1, \dots, W \quad (4)$$

$$A(\mathbf{m}^1)^T = (\mathbf{m}^2)^T + 2 \cdot \mathbf{z}, \quad (5)$$

where the objective function in Eq (3) directly minimizes the average weight distortion in Eq (2). The constraint in Eq (4) computes each effective weight from the corresponding codeword bits. The second constraint in Eq (5) ensures that the message and parity bits satisfy the constraint from Eq (1). Here, $\mathbf{z}$ is a vector of variables $[z_1, \dots, z_p]$, where each variable is a natural number between 0 and a large number M , which is used to capture the mod 2 operation in the constraint. The equations define an MILP that can be solved optimally using solvers such as CPLEX [21] and CP-SAT [7].

The outlined approach embeds n weight bits within a codeword of n bits. The method can straightforwardly be modified to embed $s = n \cdot (1 - \alpha) + \alpha \cdot (n - k)$ with $\alpha \in [0, 1]$, bits within a codeword. α represents the degree of co-embedding of the weights and the parity bits with $\alpha = 1$ represents conventional ECC. Additionally, $\alpha = 0$ is the proposed case of co-embedding all the parity bits into

weights. Whereas, $0 < \alpha < 1$ signifies that a portion of the parity bits embedded into the weights. The formulation above can be modified to incorporate α to provide a smooth trade-off between weight distortions and memory overhead. While we set α to 0 as the default value for our approach, we can scale it up and smoothly recover any nominal neural network accuracy loss for BCH codes that target high resilience.

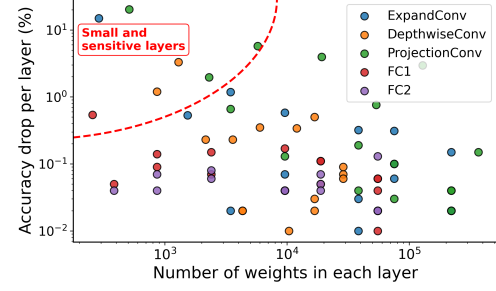


Figure 6: Layer-wise accuracy sensitivity versus weight count for EfficientNet-B0. Sensitivity is measured as the accuracy drop from perturbing each layer and plotted as point; the dotted circle highlights highly sensitive layers with relatively few weights, i.e., candidates for conventional ECC.

3.2 Hybrid ECC Schemes

In this section, we propose our hybrid ECC approach that combines conventional ECC with the proposed embedded ECC. There is a trade-off between the two approaches, embedded ECC introduces some weight distortions while conventional ECC introduces memory overhead. This trade-off could be explored using the parameter α , however, we observe below that different layers of a neural network have different sensitivity to weight distortions, which justifies the development of a hybrid ECC approach that uses different types of ECC for different layers [37].

We perform a case study to quantify the sensitivity of each neural network layer to weight distortions and size in Figure 6. The size is straightforward equal to the number of weights in the layer. The sensitivity S_l of a layer l is defined, as follows:

$$S_l = \mathbb{E}_{x \sim \mathcal{D}} [\text{Acc}(x; W) - \text{Acc}(x; W + \delta_l)], \quad (6)$$

which is the expected accuracy loss from inserting distortions δ_l into layer l of the neural network. x is a sample from the dataset $\mathcal{D}$ and $\mathbb{E}_{x \sim \mathcal{D}} [\text{Acc}(x; W)]$ is the expected model accuracy with respect to the weights W . The distortions δ_l are samples from the distribution of errors introduced by the strong BCH code (63, 39, 4). We plot the accuracy drop on y-axis against the total number of weights in the corresponding layer on x-axis as shown in Figure 6. We observe that there are a few layers that are very small and highly sensitive (top-left corner). The objective of the hybrid ECC scheme is to identify these layers and protect them using regular ECC.

The proposed hybrid framework assigns each layer a neural network to embedded ECC or conventional ECC using Algorithm 1. The algorithm provides a trade-off between inference accuracy and memory overhead. The trade-off can smoothly be explored by varying the hyperparameters τ and κ . The input to the algorithm is the number of neural network layers L , the number of weights in each layer n_l , l in $[1, L]$ and the total number of weights $N =$

Algorithm 1 Hybrid ECC Layer Assignment Algorithm

```

1: Inputs: Num. of neural network layers  $L$ ; weights per layer  $n_\ell$ 
2: Outputs: Set of layers  $C$  assigned to conventional ECC
3:  $C \leftarrow \{\}$ ;  $N \leftarrow \sum_{\ell=1}^L n_\ell$ 
4: for  $\ell \leftarrow 1$  to  $L$  do
5:   if  $n_\ell/N < \tau$  then
6:     continue
7:    $S_{\Delta\ell} \leftarrow \mathbb{E}_{x \sim \mathcal{D}} [\text{Acc}(x; W + \delta_{(A-C)}) - \text{Acc}(x; W + \delta_{(A-C-\ell)})]$ 
8:   if  $S_{\Delta\ell} > \kappa$  then
9:      $C \leftarrow C \cup \{\ell\}$ 
10: return  $C$ 

```

$\sum_{\ell=1}^L n_\ell$. Let the set C denote the layers assigned to conventional ECC. Also, let A be a set of all layers $\{1, \dots, L\}$. Initially, each layer is by default assigned to be protected using embedded ECC. The algorithm iterates through all the layers from first to last and considers assigning the layer to conventional ECC. The layer is skipped if the relative size is larger than a threshold τ . Next, we compute the expected accuracy improvement if the current layer ℓ is added to C . If the accuracy improves with more than κ , we assign the layer ℓ to C . Here, we have generalized the sensitivity equation above to be with respect to the set C instead of an unperturbed network. The set C is returned after we have iterated through all the layers of the neural network.

4 Experimental Evaluation

The experiments were performed on a CPU cluster equipped with AMD EPYC 7702 64-core processors. The experimental evaluation is performed using ResNet18 [18], ResNet50 [43], MobileNetV2 [38], EfficientNet-B0 [41] are the models trained on the CIFAR-10 [23] and ImageNet [10] datasets. Each of the models are quantized to 8-bits and all reported accuracies are top 1 accuracies. We use the BCH codes (63,57,1), (63,51,2), (63,39,4) and (63,30,6) for bit error rates (BERs) up to 1.5%, 3%, 6%, 9.5%, respectively. The burst errors are modeled by randomly injecting errors according to the BER. The parameters of hybrid partitioning algorithm are set to $\tau = 0.01$ and $\kappa = 0.1\%$. The default value for α that specifies the degree of embedded ECC is by default set to 0. Google's CP-SAT solver is used to run the embedded ECC formulation with a runtime of below a *second* for each codeword. We parallelize the embedded ECC, which allows the embedded ECC for a neural network to be computed in less than 24 hours. Note that this is a one time effort that is performed offline once per neural network. Prior methods of Weight Nulling [36], Opportunistic Parity [4], and Value-Aware Parity [25] are not publicly available and therefore re-implemented from their methodology. Value-aware Parity is implemented by overwriting the LSBs of the weights with parity information, which is an excellent baseline for our proposed approach.

4.1 Distortion minimization

We evaluate the weight distortions introduced by embedded ECC for different BER rates in Figure 7. The experiment is performed using EfficientNet-B0 on the ImageNet dataset. We observe that larger weight distortions are used for higher BER rates in Figure 7(a), as stronger BCH codes are required to be used. The average weight distortion is measured using Eq (2). Compared with opportunistic

parity [4] that fixates the ECC bits to zero and value-aware parity [25] that overwrites the LSBs, the proposed embedded ECC within E³-CODE naturally results in smaller weight distortions. This is particularly evident for stronger BCH codes that have more parity bits and provide more opportunity for weight-parity co-optimization. We assess the impact of the weight distortions on the neural network inference accuracy in Figure 7(b). The figure shows that the weight distortions of smaller magnitude result in higher inference accuracy for the network.

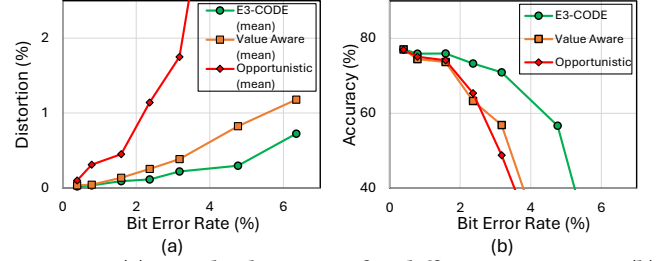


Figure 7: (a) Weight distortion for different BER rates. (b) Impact of weight distortion on inference accuracy.

4.2 Comparisons with Existing Methods

We compare the proposed E³-CODE framework against prior ECC schemes for neural networks like Weight Nulling [36], Opportunistic Parity [4], and Value-Aware Parity [25]. We evaluate two versions of our proposed scheme: E³-CODE only includes the embedded parity proposed in Section 3.1 and E³-CODE also includes the hybrid ECC scheme proposed in Section 3.2, where the former method has zero memory overhead and the latter introduces up to 2% memory overhead. We compare the inference accuracy under different BERs in Table 2.

It can be observed that all methods have a good error tolerance for BERs of 1.5%. However, the performance of weight nulling [36] starts to rapidly decline for larger BER rates. Compared with weight nulling, it can be observed that opportunistic parity [4] suffers a more smooth performance degradation in the presence of higher error rates. For this method, the errors stem from modifying the weights to set all parity bits equal to zero. The burst errors are corrected by the BCH codes. Compared with opportunistic parity, value-aware parity [25] further enhances the resilience to errors by overwriting the LSBs with parity bits. However, there is still a noticeable accuracy drop for BER rates of 6%. Compared with value-aware parity, E³-CODE matches or slightly exceeds all prior schemes at BER of 1.5% and clearly outperforms them for BER of 3%, where earlier methods suffer drastic accuracy loss. In fact, E³-CODE stays close to the software baseline for BERs of 1.5% and 3%. The accuracy improvements stem from the parity-weight co-optimization strategy, which leads to weight distortions of smaller magnitude. The results show that the co-optimization strategy is much more effective in minimizing distortion at higher BER than prior methods. This effect is more pronounced on the ImageNet dataset as it has 1,000 classes and more prone to accuracy degradation when weights are distorted. However, for BERs of 6% the accuracy even for E³-CODE drops due to the weight distortions. In particular, accuracy for EfficientNet-B0 [38] on ImageNet drops to below 2%. Compared

Table 2: Comparison of E³-CODE with state-of-the-art approaches in terms of inference accuracy in the presence of burst errors for different BER rates, models, and datasets. The Norm. row shows the normalized accuracy w.r.t. the accuracy in software. We mark the best norm. value with and without memory overhead with bold and underline, respectively.

Dataset (name)	Model (Architecture)	Software Accuracy (%)	Weight Nulling [36]			Opportunistic Parity [4]			Value-Aware Parity [25]			E ³ -CODE			E ³ -CODE-H		
			BER 1.5%	BER 3%	BER 6%	BER 1.5%	BER 3%	BER 6%	BER 1.5%	BER 3%	BER 6%	BER 1.5%	BER 3%	BER 6%	BER 1.5%	BER 3%	BER 6%
CIFAR10	ResNet-18	82.0	82.0	19.0	0.05	81.7	81.4	10.9	82.0	81.2	77.5	81.8	81.8	78.8	82.0	81.9	81.3
	ResNet-50	85.2	85.0	54.1	5.5	84.8	84.0	9.1	85.0	84.7	79.5	85.1	84.6	83.4	85.1	84.8	84.0
	MobileNet-V2	83.6	83.0	8.8	0.0	82.9	79.5	9.1	83.0	81.3	41.2	83.0	82.4	59.4	83.6	83.0	76.8
	EfficientNet-B0	82.8	82.0	2.9	0.0	80.3	64.8	9.8	82.0	77.9	22.6	82.3	78.6	55.6	82.7	82.3	80.1
ImageNet	ResNet-18	69.5	69.3	10.0	0.4	69.2	67.3	0.1	69.3	68.2	59.0	69.3	68.9	65.3	69.4	69.3	67.7
	ResNet-50	76.1	75.9	18.8	0.4	75.8	73.8	0.0	75.9	75.1	65.1	75.9	75.8	70.0	76.1	75.9	74.7
	MobileNet-V2	71.5	69.9	5.5	0.0	65.8	58.1	0.1	69.9	66.0	20.9	69.9	66.7	36.9	71.2	70.5	65.5
	EfficientNet-B0	77.0	73.7	1.1	0.0	74.1	48.8	0.1	73.7	56.8	0.5	75.7	70.9	1.8	76.6	75.5	64.0
Norm.		1.00	0.98	0.19	0.01	0.97	0.88	0.06	0.98	0.94	0.58	0.99	0.97	0.72	<u>0.99</u>	<u>0.99</u>	<u>0.94</u>

with E³-CODE, E³-CODE-H is capable of maintaining high inference accuracy for all BER rates at the expense of a 2% overhead in memory by using the hybrid ECC scheme. This overhead is minor compared with using conventional ECC for all layers. The use of conventional ECC introduces memory overheads of 10.5%, 23.5% and 61.5% to handle BER rates of 1.5%, 3%, and 6%, respectively. In particular, the hybrid method boosts the accuracy from 1.79% to 64.08%, which is relatively close to the software baseline. Hence, the use of hybrid ECC allows the software accuracy to be recovered at a fraction of the memory overhead compared with regular ECC.

Table 3: Critical BER for different ECC methods on ImageNet. Improvement ratio is measured relative to no ECC baseline.

Model	Error Correction Scheme	Critical BER	Imp. Ratio
ResNet-50	No ECC	2×10^{-7}	
	Weight Nulling	2×10^{-5}	$\times 100$
	Opportunistic Parity	8×10^{-5}	$\times 400$
	Value-aware Parity	3×10^{-4}	$\times 1500$
	E ³ -CODE	7×10^{-3}	$\times 35000$
	E ³ -CODE-H	1.5×10^{-2}	$\times 75000$
EfficientNet-B0	No ECC	2×10^{-7}	
	Weight Nulling	5×10^{-6}	$\times 25$
	Opportunistic Parity	1×10^{-5}	$\times 50$
	Value-aware Parity	4×10^{-4}	$\times 2000$
	E ³ -CODE	3×10^{-3}	$\times 15000$
	E ³ -CODE-H	7×10^{-3}	$\times 35000$

4.3 Prolonged Lifetime Evaluation

We further evaluate the different ECC methods using *critical BER*, which is defined as the highest bit error rate at which the accuracy degrades by less than 0.01% from the software baseline [25], which can be used as a measure of the system lifetime. As shown in the Table 3, the critical BER without ECC is only 2×10^{-7} . For ResNet50 [43] prior Neural Network aware ECC schemes progressively improved this to 2×10^{-5} for Weight Nulling ($\times 100$), 8×10^{-5} for Opportunistic Parity ($\times 400$), and 3×10^{-4} for Value-aware Parity Insertion ECC ($\times 1500$). Our E³-CODE framework pushes the critical BER further to 7×10^{-3} ($\times 35,000$ over no ECC) due to the reduction in weight distortion of the neural network while embedding the parity within the weights. E³-CODE-H further pushes the critical BER to 1.5×10^{-2} ($75,000\times$ over no-ECC, at the expense of minor memory overhead).

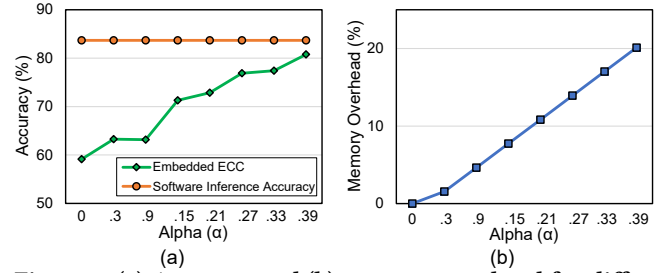


Figure 8: (a) Accuracy and (b) memory overhead for different degrees of embedding the parity bits within the weights regulated by α for MobileNetV2 on CIFAR-10 at BER of 9.5%.

4.4 High Error Tolerance Scenario

At very high BERs of 9.5%, E³-CODE-H still suffers a substantial accuracy drop to 59% from the software baseline of 83.68%. All previous studies have an accuracy of less than 1% for this scenario. It is also not very efficient to adjust the hyperparameters τ and κ to improve resilience. Instead, we mitigate the accuracy drops by varying the degree of embedded ECC regulated by the parameter α . To showcase this, we perform a case study using MobileNetV2 on CIFAR-10 at a BER of 9.5% in Figure 8. The figure shows that by varying α from 0 to 0.39, we obtain a smooth trade-off between inference accuracy in Figure 8(a) and memory overhead in Figure 8(b). By reducing the number of weight bits within each 63 bit code word with 1–13 bits, the accuracy is gradually increased to 81.13%, effectively recovering the 83.68% software accuracy. Figure 8(b) reports the corresponding memory overhead, which grows roughly linearly from about 2% to 19% as the α value increases, giving a tunable trade-off between overhead and accuracy at very high BER. This is significantly cheaper compared with conventional ECC that introduces a memory overhead of 110%.

5 Summary and Future Work

We presented E³-CODE, a post-training ECC framework for neural networks that embeds multi-bit parity directly into quantized weights. We propose an MILP-based formulation that performs the embedding while minimizing weight distortions, which translates into higher inference accuracy. Our embedded ECC scheme is extended with a hybrid variant that combines embedded ECC with

conventional ECC. By applying conventional ECC to a small set of sensitive layers, the framework can recover large portions of the accuracy loss even at high BERs. In the future, we will further explore the concept of sensitivity-aware embeddings within neural network weights. Instead of utilizing a uniform weight distortion penalty, the parity constraints can be steered so that most of the induced weight changes are absorbed by non-sensitive weights.

References

- [1] Soyed Tuhin Ahmed, Surendra Hemaram, and Mehdi B. Tahoori. 2024. NN-ECC: Embedding Error Correction Codes in Neural Network Weight Memories using Multi-task Learning. In *2024 IEEE 42nd VLSI Test Symposium (VTS)*. 1–7. doi:10.1109/VTS60656.2024.10538886
- [2] Salah A Aly, Andreas Klappenecker, and Pradeep Kiran Sarvepalli. 2007. On quantum and classical BCH codes. *IEEE Transactions on Information Theory* 53, 3 (2007), 1183–1188.
- [3] R.C. Baumann. 2005. Radiation-induced soft errors in advanced semiconductor technologies. *IEEE Transactions on Device and Materials Reliability* 5, 3 (2005), 305–316. doi:10.1109/TDMR.2005.853449
- [4] Stéphane Burel, Adrian Evans, and Lorena Anghel. 2021. Zero-Overhead Protection for CNN Weights. In *2021 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFT)*. 1–6. doi:10.1109/DFT52944.2021.9568363
- [5] P. Charpin, T. Helleseth, and V.A. Zinoviev. 2006. The coset distribution of triple-error-correcting binary primitive BCH codes. *IEEE Transactions on Information Theory* 52, 4 (2006), 1727–1732. doi:10.1109/TIT.2006.871605
- [6] Xuwei Chen, Hui Yu, and Youyun Xu. 2009. Optimized decoding schedule for irregular LDPC codes. In *IET International Communication Conference on Wireless Mobile and Computing (CCWMC 2009)*. 261–264.
- [7] Giacomo Da Col and Erich Teppan. 2019. Google vs IBM: A Constraint Solving Challenge on the Job-Shop Scheduling Problem. *Electronic Proceedings in Theoretical Computer Science* 306 (09 2019), 259–265. doi:10.4204/EPTCS.306.30
- [8] Ali Dehghan and Amir H. Banihashemi. 2018. On the Tanner Graph Cycle Distribution of Random LDPC, Random Protograph-Based LDPC, and Random Quasi-Cyclic LDPC Code Ensembles. *IEEE Transactions on Information Theory* 64, 6 (2018), 4438–4451. doi:10.1109/TIT.2018.2805906
- [9] Alexander Demidovskij and Eugene Smirnov. 2020. Effective Post-Training Quantization Of Neural Networks For Inference on Low Power Neural Accelerator. In *2020 International Joint Conference on Neural Networks (IJCNN)*. 1–7. doi:10.1109/IJCNN48605.2020.9207281
- [10] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2009. ImageNet: A Large-Scale Hierarchical Image Database. In *(CVPR)*. 248–255. https://doi.org/10.1109/CVPR.2009.5206848
- [11] Cherine Fathy and Sherine Nagy Saleh. 2022. Integrating Deep Learning-Based IoT and Fog Computing with Software-Defined Networking for Detecting Weapons in Video Surveillance Systems. *Sensors* 22, 14 (2022). doi:10.3390/s22145075
- [12] Daniel Filan, Shlomi Hod, Cody Wild, Andrew Critch, and Stuart Russell. 2020. Pruned neural networks are surprisingly modular. *arXiv preprint arXiv:2003.04881* (2020).
- [13] David Freitas, David Mota, Claiton Lopes, Daniel Simões, Jarbas Silveira, João Mota, and César Marcon. 2023. Exploration and Analysis of Combinations of Hamming Codes in 32-bit Memories. *arXiv:2306.16259 [cs.AR]* https://arxiv.org/abs/2306.16259
- [14] Ninnart Fuengfusin and Haku Tamukoh. 2024. Harden Deep Neural Networks Against Fault Injections Through Weight Scaling. *arXiv preprint arXiv:2411.18993* (2024).
- [15] Hui Guan, Lin Ning, Zhen Lin, Xipeng Shen, Huiyang Zhou, and Seung-Hwan Lim. 2019. In-place zero-space memory protection for cnn. *Advances in Neural Information Processing Systems* 32 (2019).
- [16] Piyush Gupta, Ajay Veer Chouhan, Mohammed Abdul Wajeed, Shivam Tiwari, Ankur Singh Bist, and Shiv Charan Puri. 2023. Prediction of health monitoring with deep learning using edge computing. *Measurement: Sensors* 25 (2023), 100604. doi:10.1016/j.measen.2022.100604
- [17] Song Han, Huizi Mao, and William J. Dally. 2016. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. *arXiv:1510.00149 [cs.CV]* https://arxiv.org/abs/1510.00149
- [18] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep Residual Learning for Image Recognition. In *(CVPR)*. 770–778. https://doi.org/10.1109/CVPR.2016.90
- [19] Kunping Huang, Paul H Siegel, and Anxiao Jiang. 2020. Functional error correction for robust neural networks. *IEEE Journal on Selected Areas in Information Theory* 1, 1 (2020), 267–276.
- [20] Kunping Huang, Paul H Siegel, and Anxiao Andrew Jiang. 2020. Functional error correction for reliable neural networks. In *(ISIT)*. IEEE, 2694–2699.
- [21] IBM. 2024. *IBM ILOG CPLEX Optimization Studio*. https://www.ibm.com/products/ilog-cplex-optimization-studio Accessed: 2024-11-19.
- [22] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. 2018. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2704–2713. doi:10.1109/CVPR.2018.00286
- [23] Alex Krizhevsky. 2009. Learning Multiple Layers of Features from Tiny Images. *Technical Report, University of Toronto* (2009). https://www.cs.toronto.edu/~kriz/cifar.html
- [24] Gil-Ho Kwak and Tae-Hwan Kim. 2024. Error-Resilient Binary Neural Network Inference with Selective Recompute-Based Error Correction. In *2024 21st International SoC Design Conference (ISOC)*. 364–365. doi:10.1109/ISOC62682.2024.10762521
- [25] Seo-Seok Lee and Joon-Sung Yang. 2022. Value-aware Parity Insertion ECC for Fault-tolerant Deep Neural Network. In *2022 Design, Automation and Test in Europe Conference and Exhibition (DATE)*. 724–729. doi:10.23919/DATE54114.2022.9774543
- [26] Guanpeng Li, Siva Kumar Sastry Hari, Michael Sullivan, Timothy Tsai, Karthik Pattabiraman, Joel Emer, and Stephen W. Keckler. 2017. Understanding error propagation in deep learning neural network (DNN) accelerators and applications. In *Proceedings of the international conference for high performance computing, networking, storage and analysis*. 1–12.
- [27] Jiangchen Li, Liquan Peng, Shuai Xu, and Zhixiong Li. 2024. Distributed edge signal control for cooperating pre-planned connected automated vehicle path and signal timing at edge computing-enabled intersections. *Expert Systems with Applications* 241 (2024), 122570. doi:10.1016/j.eswa.2023.122570
- [28] Yan Li. 2022. A Survey on Edge Intelligent Video Surveillance with Deep Reinforcement Learning. *Journal of Intelligence* 7 (02 2022), 70–83.
- [29] R. Logeswaran and M.U. Siddiqui. 2000. Error tolerance in classical and neural network predictors. In *2000 TENCON Proceedings. Intelligent Systems and Technologies for the New Millennium (Cat. No.00CH37119)*, Vol. 1. 7–12 vol.1. doi:10.1109/TENCON.2000.893530
- [30] A. P. Matthews. 2001. A tutorial on the basics of reed solomon and BCH error-correcting codes. *Transactions of the South African Institute of Electrical Engineers* 92, 2 (2001), 50–56.
- [31] Samir D. Mhaske, Ujjwala Ghodeswar, and G. G. Sarate. 2014. Design of area efficient Reed Solomon decoder. In *2014 2nd International Conference on Devices, Circuits and Systems (ICDCS)*. 1–4. doi:10.1109/ICDCSyst.2014.6926169
- [32] Karl C. Mohr and Lawrence T. Clark. 2006. Delay and Area Efficient First-level Cache Soft Error Detection and Correction. In *2006 International Conference on Computer Design*. 88–92. doi:10.1109/ICCD.2006.4380799
- [33] Markus Nagel, Marios Fournarakis, Rana Ali Amjad, Yelysei Bondarenko, Mart Van Baalen, and Tijmen Blankevoort. 2021. A white paper on neural network quantization. *arXiv preprint arXiv:2106.08295* (2021).
- [34] Junyong Park and Joungho Kim. 2020. Generator Polynomial Model-Based Eye Diagram Estimation Method for Bose-Chaudhuri-Hocquenghem (BCH) Code and Reed-Solomon (RS) Code. *IEEE Transactions on Electromagnetic Compatibility* 62, 1 (2020), 240–248. doi:10.1109/TEMC.2018.2881146
- [35] Flavio Ponzina, Miguel Peón-Quirós, Andreas Burg, and David Atienza. 2021. E2CNNs: Ensembles of Convolutional Neural Networks to Improve Robustness Against Memory Errors in Edge-Computing Devices. *IEEE Trans. Comput.* 70, 8 (2021), 1199–1212. doi:10.1109/TC.2021.3061086
- [36] Minghai Qin, Chao Sun, and Dejan Vucinic. 2017. Robustness of neural networks against storage media errors. *arXiv preprint arXiv:1709.06173* (2017).
- [37] Habibur Rahaman, Atri Chatterjee, and Swarup Bhunia. 2025. Secure and Storage-Efficient Deep Learning Models for Edge AI Using Automatic Weight Generation. *arXiv preprint arXiv:2507.06380* (2025).
- [38] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. 2018. MobileNetV2: Inverted Residuals and Linear Bottlenecks. In *(CVPR)*. 4510–4520. https://doi.org/10.1109/CVPR.2018.00474
- [39] Suzi Seroja Sarnin, Nani Fadzlin Nairn, and Wan Nor Syafizan W Muhamad. 2010. Performance evaluation of phase shift keying modulation technique using BCH code, Cyclic code and Hamming code through AWGN channel model in communication system. In *The 3rd International Conference on Information Sciences and Interaction Sciences*. IEEE, 60–65.
- [40] Abu Sufian, Changsheng You, and Mianxiong Dong. 2021. A Deep Transfer Learning-based Edge Computing Method for Home Health Monitoring. In *2021 55th Annual Conference on Information Sciences and Systems (CISS)*. 1–6. doi:10.1109/CISS50987.2021.9400321
- [41] Mingxing Tan and Quoc Le. 2019. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. In *(ICML)*. 6105–6114. https://proceedings.mlr.press/v97/tan19a.html
- [42] Adam Rizvi Tahiri, Mustafa Coşkun, Sultan Kübra Kılıç, and Vehbi Cagri Gungor. 2024. Intelligent traffic light systems using edge flow predictions. *Computer Standards and Interfaces* 87 (2024), 103771. doi:10.1016/j.csi.2023.103771
- [43] Dhananjay Thekedath and RR Sedamkar. 2020. Detecting affect states using VGG16, ResNet50 and SE-ResNet50 networks. *SN Computer Science* 1, 2 (2020),

- [44] Lu Wei, Chaojie Yang, Shugang Zhang, Zhong Ma, Qin Yao, and Wei Zheng. 2024. Enhancing Embedded System Performance with an Adaptive Quantization Scheme for Binary Neural Networks. In *2024 7th International Conference on Pattern Recognition and Artificial Intelligence (PRAI)*. 508–513. doi:10.1109/

- [45] Zining Zhang, Bingsheng He, and Zhenjie Zhang. 2023. Practical Edge Kernels for Integer-Only Vision Transformers Under Post-training Quantization. In *Proceedings of Machine Learning and Systems*, D. Song, M. Carbin, and T. Chen (Eds.), Vol. 5. Curran, 35–47.